

Are “Solved Issues” in SWE-bench Really Solved Correctly? An Empirical Study

You Wang
Zhejiang University
Hangzhou, China
prinzywang@zju.edu.cn

Michael Pradel
CISPA Helmholtz Center for
Information Security
Stuttgart, Germany
michael@binaervarianz.de

Zhongxin Liu*
Zhejiang University
Hangzhou, China
liu_zx@zju.edu.cn

Abstract

Automated issue solving aims to resolve real-world issues in software repositories. The most popular benchmarks for automated issue solving are SWE-bench and its human-filtered subset SWE-bench Verified, which are widely used to evaluate foundation models and software engineering agents. These benchmarks leverage testing to validate generated patches. However, because testing is rarely exhaustive, a patch may pass the tests but nevertheless fail to match the developers’ expectations. Unfortunately, it is currently unclear to what extent evaluations performed with SWE-bench suffer from such *plausible but incorrect patches*. This paper presents an in-depth empirical study of the correctness of plausible patches generated by three state-of-the-art issue-solving tools (CodeStory, LearnByInteract, and OpenHands) evaluated on SWE-bench Verified. We extensively test and inspect generated patches, and compare them against human-written ground truth patches. The core of our methodology is a novel technique for differential patch testing, called PATCHDIFF, which automatically exposes behavioral discrepancies between two patches. Our findings reveal critical weaknesses in SWE-bench’s patch validation mechanism, which causes 7.8% of all patches to count as “correct” while failing the developer-written test suite. Moreover, our novel automated technique reveals that even more (29.6%) plausible patches induce different behavior than the ground truth patches. These behavioral differences are often due to similar, but divergent implementations (46.8%) and due to generated patches that adapt more behavior than the ground truth patches (27.3%). Our manual inspection shows that 28.6% of behaviorally divergent patches are certainly incorrect. Combined, the different weaknesses lead to an inflation of reported resolution rates by 6.4 absolute percent points. Our findings are a call to arms for more robust and reliable evaluation of issue-solving tools. We envision our automated differential patch testing technique to be useful for this purpose.

CCS Concepts

• Software and its engineering;

*Corresponding Author



This work is licensed under a Creative Commons Attribution 4.0 International License. ICSE '26, Rio de Janeiro, Brazil

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2025-3/26/04

<https://doi.org/10.1145/3744916.3764576>

Keywords

Automated Issue Solving, Differential Testing, Patch Evaluation

ACM Reference Format:

You Wang, Michael Pradel, and Zhongxin Liu. 2026. Are “Solved Issues” in SWE-bench Really Solved Correctly? An Empirical Study. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26)*, April 12–18, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3744916.3764576>

1 Introduction

Automated issue solving aims to address real-world issues in software repositories, and holds great potential to reduce maintenance costs and improve software quality. SWE-bench [29] stands out as the most popular benchmark for automated issue solving, comprising 2,294 tasks from 12 well-maintained Python repositories. Each task is provided with an issue statement describing the specific task to accomplish and the repository version where the issue is to be solved. The tool to be evaluated is then required to generate a patch to address the issue. SWE-bench evaluates the generated patch by running tests associated with the issue, including at least one fail-to-pass test to confirm the issue is resolved, and, where available, several pass-to-pass tests to ensure no regression in existing functionality. Recently, OpenAI hired 93 developers to manually identify and filter out those SWE-bench tasks with overly specific and even issue-unrelated tests, and curated a subset comprising 500 tasks, known as SWE-bench Verified [3]. The SWE-bench leaderboard is getting immense attention. It has been used to judge the merit of new techniques proposed in academic papers [56, 63], and also to evaluate commercial tools for assessing their potential value [4, 6, 11]. Moreover, SWE-bench Verified has been widely used to evaluate the coding abilities of state-of-the-art foundation models, such as OpenAI GPT-o1 [7] and Anthropic Claude-3.5 [2].

However, due to practical limitations, test suites are rarely exhaustive and often suffer from weaknesses [50]. Following the literature on automated program repair [36], we refer to patches that pass their corresponding validation process as *plausible patches*. Validation with weak test suites can result in *plausible but incorrect patches*, inflating the performance of the evaluated tools, and possibly leading to incorrect conclusions about their abilities. Although others have noticed the problem of weak test suites in SWE-bench [3, 13], it is unclear to what extent evaluations performed with SWE-bench suffer from this problem, and there is a notable lack of effective methodologies for detecting plausible but incorrect patches.

This paper conducts an in-depth empirical study of the correctness of plausible generated patches on SWE-bench. We focus on the high-quality, human-filtered, and widely used subset SWE-bench Verified, and conduct this study on the plausible patches generated by the state-of-the-art issue-solving tools, i.e., CodeStory [5], LearnByInteract [51], and OpenHands [54]. Our study addresses four research questions:

RQ1: What is the impact of executing all developer tests in SWE-bench? We first analyze the validation process of SWE-bench and identify a flaw that weakens the test suites. Specifically, when validating a generated patch, SWE-bench only uses the developer-written test files modified in the pull request (PR) for fixing the target issue, potentially leaving functionality covered by other test files untested. To quantify the impact of this flaw, we execute all available test files in the corresponding repository to re-validate each patch. The results show that, on average, 7.8% of plausible patches are incorrect, leading to an absolute drop of the issue resolution rate of 4.5%, on average. This indicates that neglecting the test files not modified in the PR weakens the test suite and can inflate reported performance.

The findings from RQ1 further raise two critical questions: Are there any patches that pass all developer tests but remain incorrect? If yes, how many? Answering these questions is non-trivial. One option is to manually inspect generated plausible patches and compare them with their corresponding developer-written ground truth patch (hereon, *oracle patches*). However, this approach is labor-intensive, error-prone, and does not scale. Another option is to generate more regression tests based on the fixed repository version to strengthen the validation [57, 62]. However, this approach typically generates plenty of test cases with most of them unrelated to the generated patch, suffering from limited effectiveness. Moreover, according to our investigation (Section 4.2.3), there is a lack of robust and effective test generation tools for real-world Python projects.

To enable this study, we present a novel differential patch testing technique, named PATCHDIFF, which aims to generate test cases that expose behavioral discrepancies between the plausible patch and the developer-written oracle patch. We refer to such generated tests as *differentiating tests*. PATCHDIFF leverages Large Language Models (LLMs) for test generation and is enabled by a call-trace-based method to identify appropriate target functions and construct useful contextual information for LLMs. Specifically, we first leverage PATCHDIFF to generate differentiating tests for each plausible patch and identify behaviorally divergent patches, and then manually assess the correctness of these suspicious patches with the help of the differentiating tests. This approach provides concrete evidence of invalid functionality, enables more focused and objective patch validation, and reduces manual validation effort.

With the help of PATCHDIFF, we address the following research questions:

RQ2: How many generated plausible patches exhibit behavioral discrepancies compared to their oracle patches? To answer this question, we leverage PATCHDIFF to generate differentiating tests for the plausible patches assessed in RQ1. Our findings reveal that, on average, 29.6% of plausible patches can be differentiated from their oracle patches through the tests generated by PATCHDIFF. We refer to such patches as *suspicious patches*. This

highlights that a substantial proportion of plausible patches are likely to diverge from the expected behavior, raising concerns about their correctness.

RQ3: What are the patterns of differences between plausible and oracle patches that lead to behavioral discrepancies?

Understanding these patterns offers valuable insights into how suspicious patches deviate from their oracle patches and can inspire the development of more human-aligned issue-solving tools. To answer this question, we sample 77 (30%) suspicious patches from those identified in RQ2, manually compare them with their oracle patches, and craft a taxonomy of the patch differences leading to behavioral discrepancies. Our analysis reveals that behavioral discrepancies between plausible and oracle patches are often due to similar but divergent implementations (46.8%) and due to plausible patches adapting more behavior than their oracle patches (27.3%).

RQ4: In cases of behavioral discrepancies, how many generated plausible patches are incorrect? A behavioral discrepancy reveals incorrectness only when the behavior of the generated patch violates the expected behavior. Therefore, the correctness of a suspicious patch requires further validation. To answer this question, we manually assess the correctness of each suspicious patch evaluated in RQ3 based on its differentiating tests, developer-written tests, the repository, the issue statement, and the oracle patch. We find that 28.6% of suspicious patches are certainly incorrect. If we assume that incorrect patches are distributed evenly among suspicious patches, this result extrapolates to an approximated incorrectness rate of 11.0% among plausible patches, which inflates the resolution rates of the studied tools by 6.4 absolute percent points, on average. This result further raises concerns about the reliability of SWE-bench’s validation mechanism.

Our findings provide actionable insights for users and maintainers of issue-solving benchmarks, such as carefully selecting developer tests for patch validation, checking and filtering out plausible but incorrect patches for more accurate evaluation, and paying more attention to supplementary semantic changes in plausible patches. In addition, the under-specified issue statements in SWE-bench Verified call for better issue-solving tools that are capable of detecting and refining vague specifications and better issue-solving benchmarks where issues are well specified. We also envision PATCHDIFF to be useful for sustainably strengthening issue-solving benchmarks. Specifically, practitioners can use the differentiating tests generated by PATCHDIFF to ease their check for incorrect patches. The generated tests that successfully identify plausible but incorrect patches can be incorporated into the test suites in benchmarks. Over time, as test suites continue to evolve and become comprehensive, fewer manual efforts are required and a sustainable and robust patch validation ecosystem for issue-solving tools can be built.

In summary, the main contributions of this paper are as follows:

- *In-depth study.* The first in-depth study on the correctness of generated plausible patches on SWE-bench.
- *Technique.* A novel differential patch testing technique PATCHDIFF that can generate tests to reveal meaningful behavioral differences between patches.
- *Insights.* Insights for users and maintainers of issue-solving benchmarks towards more robust and sustainable evaluation.

Problem Statement:
<pre> simplify gives 'Imaginary coordinates are not permitted.' with evaluate(False) ## Issue 'with evaluate(False)' crashes unexpectedly with 'Point2D' ## Code import sympy as sp with sp.evaluate(False): sp.S('Point2D(Integer(1),Integer(2))') </pre>
Plausible Patch (Generated by CodeStory)
<pre> - if any(a.is_number and im(a) for a in coords): - raise ValueError('Imaginary coordinates are not permitted.') + if evaluate: + if any(a.is_number and im(a) for a in coords): + raise ValueError('Imaginary coordinates are not permitted.') </pre>
Oracle Patch
<pre> - if any(a.is_number and im(a) for a in coords): + if any(a.is_number and im(a).is_zero is False for a in coords): + raise ValueError('Imaginary coordinates are not permitted.') </pre>
Differentiating Test Generated by PatchDiff
<pre> def test_point2d_evaluate_false_with_complex_coordinate_patch_2(): > point = Point2D(1, 1, evaluate=False) E ValueError: Imaginary coordinates are not permitted. </pre>

Figure 1: An example of plausible but incorrect patches from the issue sympy-22714, where the exception is correctly raised only under the oracle patch

- *Replication package.* A replication package [8] including the implementation of PATCHDIFF and the results of our study.

2 Background and Motivating Example

This section first introduces SWE-bench and SWE-bench Verified and then describes a motivating example.

2.1 SWE-bench and SWE-bench Verified

SWE-bench [29] is the most popular benchmark for automated issue solving. As described in Section 1, to assess the effectiveness of an issue-solving tool on SWE-bench, the user needs to generate patches for each task based on the corresponding issue statement and the buggy repository version. SWE-bench also provides a test patch for each task, which includes the changes made to test files in the PR that resolves the corresponding issue. The full set of SWE-bench is shown to consist of low-quality and noise instances [3]. OpenAI curated a high-quality subset known as SWE-bench Verified, which consists of 500 samples.

To validate the correctness of a generated patch, both SWE-bench and SWE-bench Verified run the test files modified in the test patch after applying both the test patch and the generated patch to the buggy repository version. If these test files all pass, the patch is regarded as correct [9]. Although such test files are likely to be relevant to the issue, they do not necessarily cover all the functionalities that can be affected by the generated patches. As a result, this validation process is weak and may accept incorrect patches that violate the uncovered functionalities.

2.2 Motivating Example

Figure 1 presents an example of a plausible but incorrect patch generated by CodeStory. This patch attempts to address the issue that, when a `Point2D` object is created under `evaluate(False)`, the program incorrectly raises a `ValueError` with the message "Imaginary coordinates are not permitted", even though there are no imaginary inputs. To assess the correctness of this patch, we manually

compare the implementation of the two patches. The generated plausible patch modifies the safety-checking code by introducing a condition `if evaluate:`, and the oracle patch replaces `im(a)` with `im(a).is_zero()`. However, this does not directly give us enough information to determine the correctness of the generated patch. We have to read the definitions of `im()` and `is_zero()` in the repository to fully understand the issue. In fact, the underlying bug is caused by the method `im()` consistently returning a truthy value when `evaluate` is `False`, regardless of the input. The oracle patch resolves the issue by explicitly invoking `im().is_zero()` to determine the presence of imaginary numbers, while the generated patch simply mutes the examination of imaginary inputs when `evaluate` is `False`, allowing an invalid object creation to proceed. This suggests that the generated patch fails to correctly locate and fix the bug, and thus should be an incorrect patch.

The weak test suite used by SWE-bench for this issue leads to such plausible but incorrect, which leads to performance over-estimation and can cause misleading comparisons between tools. Manually identifying such plausible but incorrect patches is challenging and labor-intensive, and the manual efforts are not reusable. That is, another manual review is required if another similar but not identical plausible patch is generated. Providing tests that can expose meaningful behavioral differences between the plausible and oracle patches can significantly ease the assessment of patch correctness. In the above example, the differentiating test generated by PATCHDIFF intentionally attempts to create a `Point2D` object with imaginary coordinates while setting `evaluate` to `False`. The oracle patch correctly triggers an exception, preventing the creation of the invalid object, whereas the suspicious patch fails to trigger this error. This test directly demonstrates that the generated patch is incorrect, reducing manual efforts. Moreover, it can be added to the test suite to strengthen the patch validation of this issue.

3 Methodology

This section describes the methodology of our study, including the issue solving tools and patches we use and our differential patch testing technique.

3.1 Issue Solving Tools and Patches

Our empirical study focuses on SWE-bench Verified, a high-quality, human-validated subset of SWE-bench. The study is conducted on the plausible patches generated by three state-of-the-art issue-solving tools, i.e., CodeStory Midwit Agent + `swe-search` [5] (hereinafter, CodeStory), Learn-by-interact [51] (hereinafter, LearnBy-Interact), and OpenHands + CodeAct v2.1 [54] (hereinafter, OpenHands), on SWE-bench Verified. These tools are selected because they open source their implementations and/or release their papers or technical reports, making them suitable for further research. Note that for each tool and each issue in SWE-bench Verified, there is at most one plausible patch.

3.2 Differential Patch Testing

To enable this study, we propose an automated differential patch testing technique named PATCHDIFF. Given an issue-solving task providing the issue statement and the buggy repository version, its test patch P_t , its oracle patch P_o , and a patch P_g generated for

it, PATCHDIFF leverages an LLM to generate a set of tests that can reveal the behavioral differences between P_g and P_o , i.e., *differentiating tests*. We refer to the repository version with only P_t applied as R_t and with both P_t and P_g or P_o applied as R_g or R_o , respectively. PATCHDIFF first checks whether P_g and P_o are syntactically identical without considering comments, and omits identical P_g . Then, PATCHDIFF leverages a call-trace-based method to identify appropriate target functions and extract contextual code. Finally, PATCHDIFF prompts an LLM to generate and repair tests for target functions, and filters out unqualified tests.

3.2.1 Target Function Identification. To generate tests, PATCHDIFF first needs to determine the target function to be tested. It regards a function that satisfies the following criteria as a target function: (1) The function is a patch-modified function or a function that directly or indirectly invokes a patch-modified function. (2) The function is not defined within test files. (3) The function is directly invoked by developer-written tests. Such a function is related to patches and not related to tests, and developers usually have specific expectations regarding its behavior. To identify target functions, we first instrument each patch-modified function in R_g and R_o , and run all test files in R_g and R_o to collect call traces. Each call trace starts from a test function in R_g or R_o and ends at a patch-modified function. The first non-test function in each call trace satisfies the criterion mentioned above and PATCHDIFF annotates it as a target function. Different call traces can have the same target function. So a target function may correspond to multiple call traces.

3.2.2 Contextual Code Extraction. In each call trace, the functions in test files provide information about how to invoke the target function, and the functions in non-test files are either the target function or illustrate how the target function utilizes the patch-modified function. So all the functions in the call trace are useful for generating differentiating tests. For each target function, we extract the functions from the shortest call trace collected in R_g and the shortest one collected in R_o . For each of these functions, we further map it to its before-patch version in R_t based on its class and function names. The mapped functions provide the contextual information before patches are applied and are referred to as *context functions*. Only providing function definitions to the LLM may miss critical contextual information such as class information. Therefore, we extend context functions to construct contextual code. Specifically, we collect all Python files that contain the target function or at least one context/patch-modified function from R_t . We remove the functions that are not target, context, or patch-modified functions from these files. Any classes rendered empty after this deletion are also discarded. In addition, in test files, we annotate code lines where the target function is invoked with a comment to direct the LLM’s attention. All the streamlined files form the contextual code of this target function.

3.2.3 LLM-Based Test Generation. We prompt the LLM to generate differentiating tests for one target function at a time. It is costly to generate tests for many target functions with LLMs. Thus, we select at most 10 target functions for test generation. In detail, we calculate the number of non-test functions l in each collected call trace. For each target function, we assign its smallest l as its score. The 10 target functions with the smallest scores are selected. The rationale behind this selection is that a smaller l indicates a

simpler relationship between the patch-modified function and the target function, easing the LLM to discern and trigger behavioral discrepancies.

For each of the selected target functions, we construct a prompt by incorporating itself, P_o , P_g , its contextual code, and its shortest call traces obtained from R_o and R_g , respectively. The two example call traces demonstrate how a developer test exercises the target function, and how the target function eventually invokes the patch-affected functions. In the prompt, we instruct the LLM to first compare the two patches and reason how the patch-modified functions affect the target function through a chain-of-thought analysis, and then generate a new test file that (1) specifically tests the target function and (2) passes under one patch but fails under another patch. For each target function, we request the LLM to generate 10 responses in one request.

For each generated test file, if it fails to differentiate the patches and some tests in it fail under both patches, we further prompt the LLM with this test file and the test results under P_o , and instruct it to repair the tests so that they get passed on R_o . This step aims to repair the tests with erroneous implementations and the assertions with expected output different from the output of R_o . The repairing process iterates for 2 cycles.

To balance costs and effectiveness, we employ the OpenAI gpt-4o-mini-2024-07-18 model as the underlying LLM for test generation. This model is configured with a temperature setting of 1, allowing the generation of diverse outputs that enhance the likelihood of identifying meaningful behavioral discrepancies.

3.2.4 Unqualified Test Filtering. PATCHDIFF aims at generating tests for target functions to expose meaningful behavioral discrepancies. However, since LLMs do not always follow the instructions, the generated tests may not only examine the specified target function. To increase the probability of exposing meaningful behavioral discrepancies, PATCHDIFF filters out such tests. Specifically, we instrument patch-modified functions and execute each generated differentiating test to collect call traces. For each differentiating test, if there is a call trace where the function directly invoked by the test function is not a target function, we filter it out. Because this test does not only examine target functions. We further filter out flaky tests. In detail, for each remaining differentiating test, we run it under P_g and P_o for 20 times each. If it passes under one patch for all 20 times and fails under the other patch for at least one time, we regard it as valid. Otherwise, we filter out it. All the remaining differentiating tests are regarded as the output of PATCHDIFF.

4 Empirical Study

4.1 RQ1: Impact of Executing All Developer Tests

As described in Section 2, the validation process of SWE-bench (Verified) assesses patch correctness only based on modified test files, which can lead to plausible but incorrect patches. However, the severity of this flaw remains unclear. In this RQ, we aim to systematically evaluate the impact of this flaw on the reported performance of issue-solving tools.

Approach: To answer RQ1, for each generated plausible patch, we first apply the test patch to the repository and collect all available test files from the repository. Next, we run these test files sequentially after applying the generated patch or the oracle patch separately. We then compare the test results of the two patches and record any generated patch that shows inconsistent behavior with the corresponding oracle patch, i.e., at least one test passes with the oracle patch but fails with the generated patch. We further execute the tests that trigger inconsistent behaviors 20 times with the oracle patch to filter out flaky tests. We observe that some developer tests focus on coding conventions rather than functionality, e.g., detecting trailing whitespace or ensuring files end with no more than one newline. To prioritize functional correctness, we exclude the generated patches that only show inconsistencies related to coding conventions. The remaining generated patches introduce regression errors and are therefore incorrect.

Table 1: Incorrect patches detected by running all developer tests

Tool	%Resolved	#Incorrect	Updated %Resolved
CodeStory	62.2% (311/500)	26 (8.4%)	57.0% (↓5.2%)
LearnByInteract	60.2% (301/500)	23 (7.6%)	55.6% (↓4.6%)
OpenHands	53.0% (265/500)	19 (7.2%)	49.2% (↓3.8%)

Results: Executing all available developer tests exposes notable overestimation in the reported performance of issue-solving tools. Among the plausible patches generated by the three tools, 7.2% to 8.4% of them are functionally incorrect when subjected to all developer tests. This translates to an absolute drop of 3.8% to 5.2% in reported resolution rates, highlighting the systematic overestimation inherent in the current validation process. These findings confirm the existence of plausible but incorrect patches on SWE-bench Verified and underscore the necessity of leveraging all available developer tests to enable a more robust and comprehensive assessment of patch correctness.

An example is the plausible patch produced by CodeStory to resolve django-13279. This issue is to use legacy encode function to decode session data when DEFAULT_HASHING_ALGORITHM is set to sha1. Although the generated patch passes the tests in the PR-modified test files, it fails on another developer test where the `_legacy_decode` function is tested, suggesting that the implementation of legacy encode in the generated patch is not compatible with the original `_legacy_decode` function.

Answers to RQ1: Executing all developer tests reveals that on average 7.8% of plausible patches are incorrect, which leads to an absolute performance drop of 4.5% on average. This emphasizes the necessity of leveraging all developer tests for more robust patch validation.

4.2 RQ2: Revealing Behavioral Discrepancies Between Plausible and Oracle Patches

4.2.1 Differential Patch Testing with PATCHDIFF. The findings in RQ1 indicate that the test suites used in SWE-bench’s validation process is weak. Although the flaw mentioned in RQ1 is easy to fix, it remains unclear whether using all developer tests is good

enough to avoid plausible but incorrect patches. To investigate this question, our insight is that if a plausible patch is incorrect, it must behave differently from its oracle patch in some scenarios. Thus exposing and analyzing behavioral discrepancies between plausible patches and their corresponding oracle patches can facilitate the identification and understanding of plausible but incorrect patches.

Approach: We employ PATCHDIFF to generate tests to reveal behavioral discrepancies between each generated plausible patch and its corresponding oracle patch. As discussed in Section 3.2.1, we focus on the tests covering target functions. We refer to the tests generated by PATCHDIFF as *differentiating tests*, and the plausible patches with differentiating tests as *suspicious patches*.

Table 2: The number and the impact of the suspicious patches generated by CodeStory, LearnByInteract, and OpenHands.

Tool	%Resolved	#Patches		%Resolved w/o Susp.
		Susp.	Uniq.	
CodeStory	62.2% (311)	91 (29.3%)	74/91	44.0% (↓18.2%)
LearnByInteract	60.2% (301)	97 (32.2%)	81/97	40.8% (↓19.4%)
OpenHands	53.0% (265)	72 (27.2%)	60/72	38.6% (↓14.4%)

Susp. refers to suspicious. **Uniq.** patches refer to the suspicious patches not identified by running all developer tests.

Results: Table 2 presents the number of suspicious patches generated by each evaluated tool. Based on the tests generated by PATCHDIFF, on average 29.6% of the plausible patches are identified as suspicious patches, indicating that a substantial portion of plausible patches exhibit behavioral discrepancies from their oracle patches. If we filter out suspicious patches, the resolution rates of the three tools drop by 17.3%, on average. Notably, although the resolution rate of LearnByInteract (60.2%) is higher than that of OpenHands (53.0%), it also generates more suspicious patches than OpenHands (97 vs. 72), significantly reducing their difference in resolution rates (from 7.2% to 2.2%). This further raises concerns about the robustness of SWE-bench. Moreover, among the suspicious patches generated by each tool, on average 82.7% of them cannot be identified by running all developer tests, indicating that the generated differentiating tests complement developer tests regarding identifying suspicious patches.

Table 3: API costs of PATCHDIFF under different repair iteration bounds

Max Repair Iter	#Susp. Patches	Cost/Patch(\$)	Total Cost (\$)
2	260/877 (29.6%)	0.105	91.716
1	250/877 (28.5%)	0.075	66.186
0	228/877 (26.0%)	0.039	33.962

Table 3 details the API costs of PATCHDIFF. To get more insightful results in the empirical study, PATCHDIFF repairs the failed tests for up to 2 times, leading to a cost of 0.105\$ per patch. A lower repair iteration bound can reduce the cost to a minimum of 0.039\$ per patch, while only introducing an acceptable decrease (3.6%) of suspicious patches. These results indicate that PATCHDIFF is both effective and cost-efficient for large-scale use.

Table 4: Detected suspicious patches using different underlying LLMs (sampled 100 patches for each tool)

Model	CodeStory	LearnByInteract	OpenHands	Overall
GPT-4o-mini	26 (26.0%)	32 (32.0%)	26 (26.0%)	84 (28.0%)
DeepSeek-V3	34 (34.0%)	43 (43.0%)	40 (40.0%)	117 (39.0%)
Qwen3-A22B	45 (45.0%)	49 (49.0%)	49 (49.0%)	143 (47.7%)

4.2.2 Evaluating PATCHDIFF with Alternative LLMs. To further demonstrate that PATCHDIFF is not tied to a specific underlying LLM, we also evaluate its performance with two open-source models, deepseek-v3 and qwen3-235b-a22b-instruct-2507. For this purpose, we randomly sample 100 plausible patches from each of the three evaluated tools and apply PATCHDIFF using these models.

Results: Table 4 presents the results. When powered by more capable LLMs, PATCHDIFF achieves substantially higher detection rates of suspicious patches, showing that its effectiveness generalizes across different underlying models. This confirms that PATCHDIFF is not coupled with GPT-4o-mini and can work with open-source models. Nevertheless, for the main study, we adopt GPT-4o-mini as the underlying model because it offers both low cost and high usability.

4.2.3 Comparison with Existing Test Generators. The tests generated by existing test generators may also reveal behavioral differences between patches. However, these generators aim to generate regression tests to cover as much of the code under test as possible, while our approach targets patch-modified code and focuses on revealing behavioral discrepancies between patches. We compare PATCHDIFF with existing test generators to evaluate its effectiveness.

Approach: We use two representative test generators for Python (i.e., Pynguin [39] and CoverUp [45]), along with an issue reproduction tool LIBRO [31]. Pynguin is the state-of-the-art search-based test generation tool for Python. CoverUp is a representative LLM-based test generation tool with its implementation publicly available and easy to use. Please note that both Pynguin and CoverUp support only Python 3.10 or higher, a requirement met by only 14.8% of the 500 tasks in SWE-bench Verified. Consequently, our evaluation is confined to the plausible patches generated by the three tools that are under a compatible Python version, yielding 133 plausible patches. For both test generators, we designate the patch-modified files as the target modules, and tests are generated on the repository with the oracle patch applied. Since CoverUp supports only PyTest and fails on the instances employing other test frameworks (e.g., Django’s customized framework), we do not provide original tests to CoverUp to ensure it can handle the 133 instances. Since LIBRO was originally implemented in Java, we re-implemented it in Python. We omit its selection phase, as it only ranks tests for developers and does not potentially increase differentiating tests; instead, we execute all generated tests to check for behavioral discrepancies between each plausible patch and its oracle. We use the best-performing configuration from the original paper (two examples, $n = 50$) [31]. For both CoverUp and LIBRO, we follow PATCHDIFF’s setting and adopt gpt-4o-mini-2024-07-18 as the underlying LLM for fair comparison.

Results: Table 5 presents the evaluation results for Pynguin and CoverUp. Pynguin produces at least one test file for only 3 plausible

Table 5: Comparing PATCHDIFF with test generation tools. Gen. Tests and Diff. Tests refer to generated tests and differentiating tests, respectively.

Tool	#Patches /w Gen. Tests	#Patches /w Diff. Tests
PATCHDIFF	117 / 133	56 / 133
Pynguin	3 / 133	0 / 133
CoverUp	40 / 133	0 / 133

patches because it frequently encounters exceptions during generation. For example, for 93 patches, Pynguin raises an `AttributeError` which states that in the file `pynguin/testcase/execution.py`, attribute `IN` cannot be found in enum `Compare`, which may be a bug in Pynguin. CoverUp produces at least one test file only for 40 patches due to the difficulties it has in constructing appropriate execution environments (such as the database setup required by Django). None of the tests generated by Pynguin or CoverUp reveal behavioral discrepancies between the plausible and oracle patches, i.e., none of these tests are differentiating tests. In contrast, PATCHDIFF produces at least one test file for 117 patches and successfully generates differentiating tests for 56 of them. Note that among the 133 plausible patches, PATCHDIFF only attempts to generate tests for 117 of them. 15 of them are identical to their corresponding oracle patches and are omitted by PATCHDIFF, and 1 patch contains syntax errors and cannot be correctly parsed by the `unidiff` library [10]. Table 6 summarizes the evaluation results for LIBRO. Even under its best-performing configuration, LIBRO is able to generate differentiating tests for only 5.9% of the plausible patches, in contrast to 29.6% achieved by PATCHDIFF. This limited performance can be attributed to LIBRO’s reliance solely on issue descriptions for test generation, without considering the actual patch content. By contrast, PATCHDIFF explicitly targets the patch-relevant code and aims to generate tests that expose subtle behavioral differences between patches. These results underscore the effectiveness and necessity of PATCHDIFF in generating differentiating tests.

Table 6: Suspicious patches detected by PATCHDIFF and LIBRO

Tool	CodeStory	LearnByInteract	OpenHands	Overall
PATCHDIFF	91 (29.3%)	97 (32.2%)	72 (27.2%)	260 (29.6%)
LIBRO	18 (5.8%)	20 (6.6%)	14 (5.3%)	52 (5.9%)

Answers to RQ2: PATCHDIFF generates differentiating tests for 29.6% of the plausible patches. This suggests that a significant proportion of plausible patches behave differently from the oracle patches. In contrast, test generation baselines can only generate differentiating tests for up to 5.9% of the plausible patches.

4.3 RQ3: Patterns of Patch Differences Leading to Behavioral Discrepancies

Behavioral discrepancies revealed by generated differentiating tests provide evidence for assessing the correctness of plausible patches. This RQ aims to investigate the patterns of the differences between patches that lead to behavioral discrepancies. The answer to this RQ

can help us better understand the cause of the observed behavioral discrepancies and provide insights into developing more robust issue-solving tools.

Approach: We sample 77 (30%) of the suspicious patches detected in RQ2 for investigation and manually derive a taxonomy of patch difference patterns leading to behavioral discrepancies.

To analyze such patch difference patterns, we need to identify the commonalities and differences between each suspicious patch and its oracle patch. However, this comparison is non-trivial, because the two patches tend to have divergent syntactical implementations. To address this, we introduce the concept of atomic semantic changes (called, *sem-changes* for brevity). Sem-changes refer to a set of code line edits in a patch that alter program execution behaviors by adding or modifying a specific behavior (e.g., handling errors under certain conditions, processing specific input types or special cases, or implementing the algorithm to compute a value), as opposed to purely structural or syntactical changes. These sem-changes directly impact whether and how the patch addresses the requirements of the targeted issue. Sem-changes that add or modify the same behavior (maybe in different ways and may not be semantically equivalent) are manually aligned between the suspicious patch and the oracle patch, providing a foundation for identifying the key patch differences that lead to the observed behavioral discrepancies. Figure 2 illustrates an example of aligned sem-changes. In this case, the two diff hunks in the generated and oracle patches form two sem-changes that modify the same behavior, i.e., when the variable `arg` is of type `tuple`, an extra comma is added before the closing parenthesis at the end of the returned string. Thus they are aligned although these edits are syntactically different and occur at different positions.

The first author (A1) performs the labeling process. For any case with uncertain categories, A1 discusses with another author (A2) and reaches a consensus as the final result. Specifically, for each suspicious patch, we first read it and its corresponding oracle patch, making our best effort to understand how this two patches try to solve the issue based on their context in the repository. Then, we extract sem-changes in the two patches that add or modify the same program behavior and align them. It is possible that a sem-change in one patch does not align with any sem-change in another, i.e., it is unaligned. If there is no alignment established between the patches, the patch difference pattern is classified as ❶ *No Alignment*. Otherwise, we further review the differentiating tests of this suspicious patch and the different testing outputs under the two patches, and then determines which sem-change directly leads to the observed behavioral discrepancies. We refer to such change as *root change*. Based on the root change, the patch difference is classified into one of the three categories: ❷ *Supplementary Sem-Change*, ❸ *Absent Sem-Change*, and ❹ *Divergent Implementations of Sem-Change*. We further manually analyze the patches under the category of Supplementary Sem-changes, and classify them into subcategories based on the program behavior that the root change is adding or modifying. Theoretically, there can be cases where the observed behavioral discrepancies are attributed to hybrid patch difference patterns. However, we do not observe such cases in the sampled suspicious patches, possibly because the generated plausible patches typically correspond to issues that are not very complex. Therefore, we exclude hybrid patterns from our discussion.

Table 7: Taxonomy for patterns of patch differences that lead to behavioral discrepancies. Sem-change refers to atomic semantic change.

Category	#Cases
Total	77
Absent Sem-Change	4 (5.2%)
Supplementary Sem-Change	21 (27.3%)
– Explicitly Handling More Possible Situations	13 (16.9%)
– Supplementary Change of Application Logics	8 (10.4%)
Divergent Implementations of Sem-Change	36 (46.8%)
No Alignment	16 (20.8%)

Results: Table 7 presents our derived taxonomy of patch difference patterns. We elaborate on each pattern as follows:

❶ *No Alignment (20.8%)*. This pattern refers to the cases where there is no alignment of automatic semantic changes between the suspicious and oracle patches. Figure 3a illustrates an example. In this example, the suspicious patch changes the base class of `IsNull` and adds a new class member, while the oracle patch modifies the condition under which an error should be appended to the errors list. The two changes are not aligned.

❷ *Supplementary Sem-Change (27.3%)*: This pattern refers to the cases where an unaligned Sem-change in the suspicious patch directly leads to the observed behavioral discrepancies. In other words, the suspicious patch alters the program behavior in a way that the oracle patch does not. Figure 3b illustrates an example, where the change of handling input 0 in `_check_vector` does not align with any change in the oracle patch. This category is further divided into two subcategories: *Explicitly Handling More Possible Situations (16.9%)*, where a behavior added only in the generated patch leads to the observed behavioral discrepancies (e.g., explicitly handling inputs of special cases, extra safety checks); and *Supplementary Change of Application Logics (10.4%)*, where a behavior modified only in the generated patch leads to the observed behavioral discrepancies (e.g., modifying the algorithm to calculate a value, which stays unchanged under the oracle patch).

❸ *Absent Sem-Change (5.2%)*. This is the opposite of the second category, where an unaligned sem-change in the oracle patch directly leads to the observed behavioral discrepancies. Figure 3c presents an example. In this example, the sem-changes of preventing deprecation warning when values is empty in function `convert` are aligned, while an unaligned change in oracle patch which modifies another function update is not found in the suspicious patch.

❹ *Divergent Implementations of Sem-Change (46.8%)*. In this case, different implementations of aligned sem-changes directly lead to the observed behavioral discrepancies. Figure 2 illustrates an example. In this case, the left sem-change only adds the comma when the `arg` is not empty, while the right one always adds the comma. Note that the upper part of the oracle patch is for refactoring, and thus is not a sem-change.

We can see from Table 7 that behavioral differences between suspicious and oracle patches are often due to *Divergent Implementations of Sem-Change (46.8%)* and due to *Supplementary Sem-Change*

Suspicious Patch (Generated by LearnByInteract)	Oracle Patch
<pre> sympy/utilities/lambdify.py @@ -961,7 +961,12 @@ def _recursive_to_string(doprint, arg): else: raise NotImplementedError("unhandled type: %s, %s" % (type(arg), arg)) - return left + ', '.join(_recursive_to_string(doprint, e) for e in arg) + right + elements = [_recursive_to_string(doprint, e) for e in arg] + joined = ', '.join(elements) + if isinstance(arg, tuple) and len(arg) == 1: + joined += ', ' + return left + joined + right elif isinstance(arg, str): return arg </pre>	<pre> sympy/utilities/lambdify.py @@ -956,9 +956,9 @@ def _recursive_to_string(doprint, arg): elif iterable(arg): if isinstance(arg, list): - left, right = "[", "]" + left, right = "[", "]" elif isinstance(arg, tuple): - left, right = "(", ")" + left, right = "(", ")" else: raise NotImplementedError("unhandled type: %s, %s" % (type(arg), arg)) return left + ', '.join(_recursive_to_string(doprint, e) for e in arg) + right </pre>

Figure 2: Example of aligned sem-changes (sympy-23262)

Suspicious Patch (Generated by LearnByInteract)	Oracle Patch
<pre> django/db/models/lookups.py @@ -485,17 +485,13 @@ @Field.register_lookup -class IsNull(BuiltinLookup): +class IsNull(Transform): lookup_name = 'isnull' + output_field = BooleanField() prepare_rhs = False </pre>	<pre> django/db/models/base.py @@ -1747,7 +1747,9 @@ def _check_ordering(cls): _cls = None except (FieldDoesNotExist, AttributeError): - if fld is None or fld.get_transform(part) is None: + if fld is None or (+ fld.get_transform(part) is None and fld.get_lookup(part) is None +): errors.append(checks.Error(</pre>

(a) No alignment (django-12858)

Suspicious Patch (Generated by OpenHands)	Oracle Patch
<pre> sympy/physics/vector/vector.py @@ -57,6 +57,8 @@ def __add__(self, other): """The add operator for Vector. """ + if other == 0: + return self other = _check_vector(other) return Vector(self.args + other.args) ... @@ -721,6 +726,8 @@ def _check_vector(other): + if other == 0: + return Vector(0) if not isinstance(other, Vector): raise TypeError('A Vector must be supplied') </pre>	<pre> sympy/physics/vector/vector.py @@ -57,6 +57,8 @@ def __add__(self, other): """The add operator for Vector. """ + if other == 0: + return self other = _check_vector(other) return Vector(self.args + other.args) </pre>

(b) Supplementary sem-change (sympy-14711)

Suspicious Patch (Generated by LearnByInteract)	Oracle Patch
<pre> lib/matplotlib/category.py @@ -53,17 +53,21 @@ def convert(value, unit, axis): values = np.atleast_1d(np.array(value, dtype=object)) + if values.size == 0: + return np.array([], dtype=float) with _api.suppress_matplotlib_deprecation_warning(): is_numlike = all(units.ConversionInterface.is_numlike(v) and not isinstance(v, (str, bytes)) for v in values) if is_numlike: _api.warn_deprecated(</pre>	<pre> lib/matplotlib/category.py @@ -58,7 +58,7 @@ def convert(value, unit, axis): for v in values) if is_numlike: if values.size and is_numlike: _api.warn_deprecated("3.5", message="Support for passing numbers through unit ") @ -230,7 +230,7 @@ def update(self, data): self._mapping[val] = next(self._counter) - if convertible: + if data.size and convertible: _log.info('Using categorical units to plot a list of strings ' </pre>

(c) Absent sem-change (matplotlib-22719)

Figure 3: Three examples of patch difference patterns

(27.3%). Interestingly, there are much more suspicious patches in *Supplementary Sem-Change* than in *Absent Sem-Change* (27.3% v.s. 5.2%). This suggests that suspicious patches tend to introduce additional changes rather than omitting necessary changes. In addition, 16.9% of the suspicious patches in *Supplementary Sem-Change* add extra behaviors to explicitly handle more possible situations. Although these additional behaviors could make the suspicious patch more robust, they could also be unnecessary, violate user requirements, or even introduce new defects.

Answers to RQ3: Behavioral differences between suspicious and oracle patches are often due to similar, but divergent implementations (46.8%) and due to suspicious patches contain more semantic changes than the oracle patches. There are much more cases where the suspicious patches introduce supplementary semantic changes than those with absent semantic changes (27.3% vs. 5.2%).

Table 8: Results of manually validating patch correctness

Category	#Cases
Total	77
Incorrect Patches Detected in RQ1	11 (14.3%)
Incorrect Patches	22 (28.6%)
– Regressive Patches	11 (14.3%)
– Partial Fixes	6 (7.8%)
– Patches with Irrelevant Behavioral Changes	3 (3.9%)
– Patches with Erroneous Modifications	2 (2.6%)
Correct Patches	4 (5.2%)
– Irrelevant Changes in Oracle Patch	2 (2.6%)
– Invalid Differentiating Tests	2 (2.6%)
Patches with Uncertain Correctness	51 (66.2%)

4.4 RQ4: The Correctness of Suspicious Patches

Suspicious patches are not necessarily incorrect patches. For example, the program behavior with illegitimate values as inputs can be undefined. If a differentiating test triggers different behaviors with such illegitimate inputs, the corresponding suspicious patch can also be correct. In this RQ, we aim to analyze the correctness of suspicious patches and identify the reasons for correct and incorrect suspicious patches.

Approach: We utilize the suspicious patches sampled in RQ3. For each sampled suspicious patch, the first author (A1) manually analyzes the user requirements in the issue statement, the results of running all developer-written regression tests (identified in RQ1), and the results of running the generated differentiating tests, and then compares the implementations of the suspicious and oracle patches. A patch is considered correct if it satisfies the requirements specified in the issue statement without introducing errors. In cases of ambiguity, A1 discusses with another author (A2) until a consensus is reached.

Results: After manual inspection, each sampled patch is classified as incorrect, correct, or with uncertain correctness, as summarized in Table 8.

Incorrect Patches (22): We identify 28.6% of the suspicious patches as incorrect, which can further be divided into four sub-categories. ❶ *Regressive Patches (11)*: These patches successfully satisfy the requirements mentioned in the issue statements, but accidentally introduce faults into some functionalities that are unrelated to the target issues. Our motivating example in Subsection 2.2 falls into this category, where the issue is to fix the bug that the `ValueError` of "Imaginary coordinates are not permitted" is raised under with `evaluate(False)` when there is no imaginary input. The generated patch satisfies the user requirement that this `ValueError` is now not raised under ordinary inputs. However, it also makes the object creation never raise this `ValueError` when `evaluate` is set to `False`, even though there is an imaginary input. This violates the original functionality, therefore this patch falls into Regressive Patches. ❷ *Partial Fixes (6)*: These patches partially resolve the target issues but fail to satisfy the requirements in the issue statements for certain legitimate inputs. For example, the patch produced by LearnByInteract to resolve scikit-learn-14496 aims to fix a bug where the `OPTICS::fit` function

raises `TypeError` when the `min_samples` attribute of `OPTICS` is set to a float between 0 and 1. However, the patch fails to locate and fix every piece of relevant buggy code and makes `OPTICS::fit` raise `ValueError` under certain `min_samples` between 0 and 1, violating the issue statement. ❸ *Patches with Irrelevant Behavioral Changes (3)*: These patches successfully satisfy the requirements mentioned in the issue statements but also introduce some unnecessary changes. For example, the patch produced by LearnByInteract for django-15104 changes a code line to safely remove the key to from a dictionary when it exists, which successfully fixes the bug. However, this patch additionally repeats another code line `fields_def.append(deconstruction)`, which introduces duplicate items in `fields_def`. Different from *Regressive Patches*, which introduces regression in their implementation to resolve the issue, patches in this category contain changes that do not contribute to resolving the issue. ❹ *Patches with Erroneous Modifications (2)*: These patches contain clear implementation errors. For example, the patch produced by CodeStory to resolve sympy-20801 captures exceptions of type `SympifyError`, which is not defined before. These patches are plausible because the original test suites fail to cover the buggy branches and do not trigger runtime errors.

By sampling 30% of the suspicious patches for manual validation, we identify 22 incorrect patches. If we assume that the incorrect patches are distributed evenly among suspicious patches and consider the incorrect patches that are discovered in RQ1 but are not suspicious (which results in 23 patches), the estimated incorrect rate will be 11.0% among plausible patches. This leads to the resolution rates of the studied tools being inflated by 6.4 points on average, further confirming the prevalence of performance overestimation and necessitating tools like PATCHDIFF for detecting plausible but incorrect patches. In addition, 11 (50.0%) of the 22 incorrect patches cannot be identified by running all developer tests, underscoring PATCHDIFF’s effectiveness in helping identify incorrect patches. Among the 22 incorrect patches, 8 (36.4%) patches are attributed to faulty supplementary semantic changes. This highlights the risk of supplementary changes in plausible patches and emphasizes the need for issue-solving tools to rigorously review and refine the plausible patches.

Correct Patches (4): Only 5.2% patches are certainly correct, which can be divided into two sub-categories. ❶ *Irrelevant Oracle Changes (2)*: In these cases, the observed behavioral discrepancy arises from the changes in oracle patches that do not contribute to issue resolution. So there is no evidence suggesting the plausible patch fails to address the issue. ❷ *Invalid Differentiating Tests (2)*: In these cases, the differentiating tests rely on illegitimate inputs where the program’s expected behavior is undefined. Such tests should not lead to incorrect patches.

Patches with Uncertain Correctness (51): For 66.2% of the sampled suspicious patches, we cannot determine their correctness based on the information that we can find from SWE-bench and the corresponding repositories. This uncertainty arises due to the differences in implementation details between the plausible and oracle patches that are not explicitly specified in the issue statement. In such cases, neither the issue-solving tools nor our analysis can reliably infer user requirements regarding these implementation details. The plausible patch generated by CodeStory to solve matplotlib-25311 falls in this category. The issue aims

at fixing a bug where pickling figures raises "TypeError: cannot pickle FigureCanvasQTAgg object". This plausible patch introduces `__getstate__` and `__setstate__` functions to ensure that the unpickleable attribute `canvas` is set to `None` during serialization and remains `None` upon deserialization, which is a common practice for handling such issues. The oracle patch, however, transforms `canvas` into a property using a lambda expression, so that the value of `canvas` is dynamically retrieved when accessed and not stored during pickling. The key difference is that the plausible patch explicitly sets `canvas` to `None` upon loading, whereas the oracle patch makes it remain accessible after deserialization. Since there are no explicit requirements dictating whether `canvas` should remain accessible, we cannot determine the correctness of this plausible patch. While some unspecified details may have negligible or no impact, others could lead to unintended behaviors or latent issues. These findings underscore a need to enhance issue-solving tools with the capabilities to detect ambiguous or under-specified requirements and to proactively prompt users for clarification. They also imply that the community may need a better benchmark, where issues are well specified and leave less ambiguity.

Answers to RQ4: Among the manually validated 77 suspicious patches, 22 (28.6%) patches are incorrect. This interprets to an estimated incorrect rate of 11.0% in plausible patches, inflating the reported resolution rate by 6.4 points on average. 51 (66.2%) patches have uncertain correctness due to under-specified requirements.

5 Discussion

5.1 Implications

Carefully selecting developer tests for robust patch validation. As shown in RQ1, ignoring the test files not modified in the PR leads to notable performance inflation. Yet, we also find some projects contain some non-functional tests, e.g., tests related to code conventions. These findings indicate that maintainers of issue-solving benchmarks should carefully select developer tests for patch validation. One suggestion is to use all developer tests by default and exclude non-functional tests for benchmarks only focusing on functional correctness.

Awareness of plausible but incorrect patches. The results on RQ4 show that the patches that pass all developer tests can still be incorrect. This suggests that both the users and the maintainers of SWE-bench should check plausible patches generated by issue-solving tools and filter out incorrect patches for more accurate evaluation, as is common in automated program repair [32].

Paying more attention to patches introducing supplementary semantic changes. As shown in RQ4, 36.4% of the certainly incorrect patches are attributed to faulty supplementary semantic changes, suggesting that additional changes can be an indicator for incorrectness. Thus, the users and maintainers of SWE-bench should pay more attention to patches with supplementary semantic changes when checking the correctness of patches.

Handling under-specified issue statements. The result of RQ4 shows that a large proportion of suspicious patches have uncertain correctness due to under-specified requirements in the issue statement. These patches potentially introduce unintended or undesired

behaviors, compromising the robustness of the target project. This calls for better issue-solving tools that are capable of detecting and refining under-specified requirements collaboratively with users.

Building a new benchmark with well-specified statements. Although the issue statements in SWE-bench are considered to be of high quality by human annotators [3], vague issue statements with under-specified requirements still exist, as shown in RQ4. Such issue statements can misguide issue-solving tools in generating incorrect patches, constraining the reliability of the benchmark. Practitioners should consider building better benchmarks, where issues are well-specified and leave less ambiguity.

5.2 Towards Sustainable Patch Validation in SWE-bench

We envision PATCHDIFF being useful for sustainably strengthening SWE-bench and other issue-solving benchmarks. For users of SWE-bench, before submitting the patches produced by their tools to the SWE-bench leaderboard, it is advisable to assess the plausible patches by utilizing PATCHDIFF to generate differentiating tests and manually examine if the behavioral discrepancies exposed by tests lead to incorrect patches. This would lead to a more accurate evaluation. While such a practice undoubtedly imposes additional effort and cost on users, it brings significant long-term benefits. Once a user identifies plausible but incorrect patches with PATCHDIFF, she can submit the differentiating tests that expose incorrectness to the SWE-bench leaderboard. Incorporating these tests into the benchmarks's original test suite enables the identification of similar incorrect patches in the future, thus strengthening the validation process. This collaborative contribution facilitates a more rigorous validation process for subsequent users. Over time, as the test suite continues to evolve and becomes comprehensive, the additional burden would also decrease. This fosters a sustainable and robust patch validation ecosystem for assessing issue-solving tools.

5.3 Threats to Validity

Our study is subject to several potential threats to validity and intrinsic limitations: 1) To balance costs and effectiveness, the implementation of PATCHDIFF leverages GPT-4o-mini as the underlying LLM. While this choice limits access to potentially more advanced models, PATCHDIFF successfully generates differentiating tests for 29.6% of the plausible patches, establishing a solid basis for later studies. 2) The analysis conducted in RQ3 and RQ4 involves manual analysis, potentially introducing human bias. To reduce this risk, the first author collaborates with another author to resolve ambiguous cases and reach a consensus on uncertain assessments, thereby enhancing objectivity. 3) For RQ3 and RQ4, we sample 30% (77) of the suspicious plausible patches for analysis. This sample size may introduce sampling bias. However, this sample size corresponds to a confidence interval of 9.39% with a confidence level of 95%. Prior work [27, 34] shows that it is sufficiently large to draw statistically meaningful conclusions. 4) RQ3 and RQ4 are restricted to the suspicious patches identified by PATCHDIFF, potentially introducing bias as this subset may not fully represent the characteristics of all plausible patches. Despite threats 3) and 4), the observed trends and patterns still provide meaningful evidence regarding the severity

of performance overestimation and offer actionable insights into patch correctness validation.

6 Related Work

Weak test suites. The problem of weak test suites has been well studied in the area of test-based automatic program repair (APR) [37, 40, 46, 50]. For instance, Martinez et al. [40] manually assessed 84 plausible patches generated on the Defects4J benchmark [30], and found that only 11 of them are correct. Qi et al. [46] found that most of the plausible patches are equivalent to a single modification that deletes the corresponding functionality, which is problematic. Some prior work proposed actionable strategies to mitigate this problem in APR [50, 58, 61, 62]. For example, Smith et al. [50] found that using a test suite that is inaccessible to evaluated tools can help detect plausible but incorrect patches. Xiong et al. [58] proposed to incorporate execution behavior similarity under generated test inputs to detect incorrect patches that behave significantly differently from their oracle patches. For test-based APR benchmarks, test suites are provided to the evaluated tools, so plausible but incorrect patches are also called overfitting patches. Different from these studies, our work focuses on the issue-solving task, which aims to resolve issues based on issue statements rather than test suites. For issue-solving benchmarks, the evaluated tools cannot access test suites during evaluation. Thus, prior conclusions may not apply to SWE-bench. Recently, Aleithan et al. [13] also noticed the weak test suite problem in SWE-bench. However, first, their study is conducted on the patches generated by only one issue-solving tool on SWE-bench full, which is shown to contain many low-quality instances. In contrast, our study covers three state-of-the-art tools and focuses on the human-filtered subset SWE-bench Verified. Second, they focus on manually classifying plausible patches, while we dig deeper into the patterns and types of plausible but incorrect patches with our novel technique PATCHDIFF.

Automated test generation. Various automated test generation approaches are proposed to alleviate testing efforts and help find bugs [1, 26, 39, 44]. *Traditional test generation* approaches utilize some predefined rules to generate tests and can be mainly categorized into search-based [26, 39], randomization-based [44], and constraints-based [18, 52] approaches. *Deep learning-based test generation* approaches train deep learning models to generate natural and human-understandable test cases [12, 22, 47, 53]. Recently, researchers have proposed *LLM-based test generation* approaches [20, 45, 49, 55]. For example, CoverUp iteratively instructs LLMs to generate Python regression tests, improve coverage, and fix errors with detailed coverage information. Liu et al. [35] proposed to augment the test suites in the code generation benchmarks HumanEval [19] and MBPP [14] based on LLMs and type-aware mutation. These approaches aim to generate regression tests to cover as much of the code as possible, while PATCHDIFF targets differentiating two patches, which requires more targeted test generation. Differential testing aims at generating tests to expose different behaviors between two versions of a program [24, 25, 33, 38]. Mokav [24] uses execution feedback to iteratively guide an LLM in generating tests to differentiate different solutions of programming competition tasks. Existing LLM-based differential testing techniques focus on

function-level programs, while PATCHDIFF can generate differentiating tests for large and complex projects. In addition, PATCHDIFF targets differential patch testing, and leverages specially designed methods to identify appropriate target functions and to construct useful contextual information for revealing meaningful behavioral differences, which are important for differential patch testing but are not considered by prior work.

Software engineering agents. LLM-based agents promise to help automate various software engineering tasks [48]. Agents for various tasks have been presented, such as fault localization [28, 31, 59], issue solving [56, 60, 63], automated program repair [15, 21], repository setup [17, 23, 41], and generating issue-reproducing tests [42, 43]. Researchers have started to study the behavior such agents to better understand their strengths and weaknesses [16]. Our work complements such efforts and will help improve future software engineering agents by critically analyzing the patches produced by the agents.

7 Conclusion

This paper presents an in-depth empirical study of the correctness of plausible generated patches on SWE-bench. We first find that the validation process of SWE-bench overlooks non-modified test files, which leads to significant performance overestimation. Then we extensively investigate the prevalence of plausible patches that exhibit behavioral discrepancies from ground truth patches, the patch difference patterns leading to behavioral discrepancies, and the correctness of plausible patches that exhibit behavioral discrepancies. The core of our methodology is the novel PATCHDIFF technique for differential patch testing, which automatically exposes behavioral discrepancies between two patches. The results call for carefully selecting developer tests for patch validation, checking and filtering out plausible but incorrect patches for more accurate evaluation, and paying more attention to supplementary semantic changes in plausible patches. Our work will contribute toward better issue-solving tools and benchmarks that address the problem of under-specified specifications. PATCHDIFF can provide concrete evidence for invalid functionality in patches through generated tests, enabling more focused and objective patch assessment. The test generated by PATCHDIFF can be continuously used to strengthen issue-solving benchmarks with the help of benchmark users. We envision PATCHDIFF to be useful for building a sustainable and robust patch validation ecosystem for assessing issue-solving tools.

8 Data Availability

Our code and data are available: <https://github.com/ZJU-CTAG/PatchDiff>

Acknowledgments

This work was supported by the National Natural Science Foundation of China (No. 62202420), Zhejiang Provincial Natural Science Foundation of China (No. LZ25F020003), the European Research Council (ERC, grant agreements 851895 and 101155832), and the German Research Foundation within the DeMoCo project.

References

- [1] 2023. agitar. <http://www.agitar.com/>. [Online]. Available: <http://www.agitar.com/>. Accessed: 2025-09-04.
- [2] 2024. Introducing computer use, a new Claude 3.5 Sonnet, and Claude 3.5 Haiku. <https://www.anthropic.com/news/3-5-models-and-computer-use> Accessed: 2025-09-04.
- [3] 2024. Introducing SWE-bench Verified. <https://openai.com/index/introducing-swe-bench-verified/> Accessed: 2025-09-04.
- [4] 2025. BLACKBOX.AI. <https://www.blackbox.ai/> Accessed: 2025-09-04.
- [5] 2025. codestoryai/aide: The open-source AI-native IDE. <https://github.com/codestoryai/aide> Accessed: 2025-09-04.
- [6] 2025. Isoform - Get custom integrations to close B2B deals using your 24/7 integration developer. <https://www.isoform.ai/> Accessed: 2025-09-04.
- [7] 2025. OpenAI o1 and new tools for developers. <https://openai.com/index/o1-and-new-tools-for-developers/> Accessed: 2025-09-04.
- [8] 2025. Our replication package. <https://github.com/ZJU-CTAG/PatchDiff> Accessed: 2025-09-04.
- [9] 2025. SWE-bench/swebench/harness at main · swe-bench/SWE-bench. <https://github.com/swe-bench/SWE-bench/tree/main/swebench/harness> Accessed: 2025-09-04.
- [10] 2025. The unidiff library. <https://pypi.org/project/unidiff/> Accessed: 2025-09-04.
- [11] 2025. Weights & Biases: The AI Developer Platform. <https://wandb.ai/site> Accessed: 2025-09-04.
- [12] Saranya Alagarsamy, Chakkrit Tantithamthavorn, and Aldeida Aleti. 2024. A3test: Assertion-augmented automated test case generation. *Information and Software Technology* 176 (2024), 107565.
- [13] Reem Aleithan, Haoran Xue, Mohammad Mahdi Mohajer, Elijah Nnorom, Gias Uddin, and Song Wang. 2024. SWE-Bench+: Enhanced Coding Benchmark for LLMs. *arXiv preprint arXiv:2410.06992* (2024).
- [14] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021).
- [15] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2025. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair. In *International Conference on Software Engineering (ICSE)*.
- [16] Islem Bouzenia and Michael Pradel. 2025. Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories. In *ASE*.
- [17] Islem Bouzenia and Michael Pradel. 2025. You name it, I run it: An LLM agent to execute tests of arbitrary projects. *Proceedings of the ACM on Software Engineering* 2, ISSTA, 1054–1076.
- [18] Cheng-Hung Chang and Nai-Wei Lin. 2016. Constraint-based test case generation for white-box method-level unit testing. In *2016 International Computer Symposium (ICS)*. 601–604.
- [19] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [20] Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. 2024. Chatunitest: A framework for llm-based test generation. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. 572–576.
- [21] Runxiang Cheng, Michele Tufano, Jürgen Cito, José Cambroner, Pat Rondon, Renyao Wei, Aaron Sun, and Satish Chandra. 2025. Agentic Bug Reproduction for Effective Automated Program Repair at Google. *arXiv preprint arXiv:2502.01821* (2025).
- [22] Elizabeth Dinella, Gabriel Ryan, Todd Mytkowicz, and Shuvendu K Lahiri. 2022. Toga: A neural method for test oracle generation. In *Proceedings of the 44th International Conference on Software Engineering*. 2130–2141.
- [23] Aleksandra Eliseeva, Alexander Kovrigin, Ilia Kholkin, Egor Bogomolov, and Yaroslav Zharov. 2025. EnvBench: A Benchmark for Automated Environment Setup. *arXiv:2503.14443 [cs.LG]* <https://arxiv.org/abs/2503.14443>
- [24] Khashayar Etemadi, Bardia Mohammadi, Zhendong Su, and Martin Monperrus. 2024. Mokav: Execution-driven differential testing with llms. *arXiv preprint arXiv:2406.10375* (2024).
- [25] Robert B Evans and Alberto Savoia. 2007. Differential testing: a new approach to change detection. In *The 6th Joint Meeting on European software engineering conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering: Companion Papers*. 549–552.
- [26] Gordon Fraser and Andrea Arcuri. 2011. Evosuite: automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering*. 416–419.
- [27] Safwat Hassan, Chakkrit Tantithamthavorn, Cor-Paul Bezemer, and Ahmed E Hassan. 2018. Studying the dialogue between users and developers of free apps in the google play store. *Empirical Software Engineering* 23 (2018), 1275–1312.
- [28] Zhonghao Jiang, Xiaoxue Ren, Meng Yan, Wei Jiang, Yong Li, and Zhongxin Liu. 2025. CoSIL: Software Issue Localization via LLM-Driven Code Repository Graph Searching. *arXiv preprint arXiv:2503.22424* (2025).
- [29] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2023. SWE-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770* (2023).
- [30] René Just, Darioush Jalali, and Michael D Ernst. 2014. Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 2014 international symposium on software testing and analysis*. 437–440.
- [31] Sungmin Kang, Juyeon Yoon, Nargiz Askarbekzyzy, and Shin Yoo. 2024. Evaluating diverse large language models for automatic and general bug reproduction. *IEEE Transactions on Software Engineering* (2024).
- [32] Claire Le Goues, Michael Pradel, and Abhik Roychoudhury. 2019. Automated program repair. *Commun. ACM* 62, 12 (2019), 56–65. doi:10.1145/3318162
- [33] Tsz-On Li, Wenxi Zong, Yibo Wang, Haoye Tian, Ying Wang, Shing-Chi Cheung, and Jeff Kramer. 2023. Nuances are the key: Unlocking chatgpt to find failure-inducing tests with differential prompting. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 14–26.
- [34] Mario Linares-Vásquez, Gabriele Bavota, Massimiliano Di Penta, Rocco Oliveto, and Denys Poshyvanyk. 2014. How do api changes trigger stack overflow discussions? a study on the android sdk. In *proceedings of the 22nd International Conference on Program Comprehension*. 83–94.
- [35] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2024. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Advances in Neural Information Processing Systems* 36 (2024).
- [36] Kui Liu, Anil Koyuncu, Dongsun Kim, and Tegawendé F Bissyandé. 2019. TBar: Revisiting template-based automated program repair. In *Proceedings of the 28th ACM SIGSOFT international symposium on software testing and analysis*. 31–42.
- [37] Kui Liu, Li Li, Anil Koyuncu, Dongsun Kim, Zhe Liu, Jacques Klein, and Tegawendé F Bissyandé. 2021. A critical review on the evaluation of automated program repair systems. *Journal of Systems and Software* 171 (2021), 110817.
- [38] Kaibo Liu, Yiyang Liu, Zhenpeng Chen, Jie M Zhang, Yudong Han, Yun Ma, Ge Li, and Gang Huang. 2024. Llm-powered test case generation for detecting tricky bugs. *arXiv preprint arXiv:2404.10304* (2024).
- [39] Stephan Lukaszczuk and Gordon Fraser. 2022. Pynguin: Automated unit test generation for python. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*. 168–172.
- [40] Matias Martinez, Thomas Durieux, Jifeng Xuan, Romain Sommerard, and Martin Monperrus. 2015. Automatic repair of real bugs: An experience report on the defects4j dataset. *arXiv preprint arXiv:1505.07002* (2015).
- [41] Louis Milliken, Sungmin Kang, and Shin Yoo. 2025. Beyond pip install: Evaluating llm agents for the automated installation of python projects. In *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 1–11.
- [42] Niels Mündler, Mark Niklas Müller, Jingxuan He, and Martin Vechev. 2024. Code Agents are State of the Art Software Testers. *arXiv:2406.12952 [cs.SE]* <https://arxiv.org/abs/2406.12952>
- [43] Noor Nashid, Islem Bouzenia, Michael Pradel, and Ali Mesbah. 2025. Issue2Test: Generating Reproducing Test Cases from Issue Reports. *arXiv preprint arXiv:2503.16320* (2025).
- [44] Carlos Pacheco and Michael D Ernst. 2007. Randoop: feedback-directed random testing for Java. In *Companion to the 22nd ACM SIGPLAN conference on Object-oriented programming systems and applications companion*. 815–816.
- [45] Juan Altmayer Pizzorno and Emery D Berger. 2024. Coverup: Coverage-guided llm-based test generation. *arXiv preprint arXiv:2403.16218* (2024).
- [46] Zichao Qi, Fan Long, Sara Achour, and Martin Rinard. 2015. An analysis of patch plausibility and correctness for generate-and-validate patch generation systems. In *Proceedings of the 2015 international symposium on software testing and analysis*. 24–36.
- [47] Nikitha Rao, Kush Jain, Uri Alon, Claire Le Goues, and Vincent J Hellendoorn. 2023. CAT-LLM training language models on aligned code and tests. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 409–420.
- [48] Abhik Roychoudhury, Corina Pasareanu, Michael Pradel, and Baishakhi Ray. 2025. Agentic AI Software Engineer: Programming with Trust. *arXiv preprint arXiv:2502.13767* (2025).
- [49] Gabriel Ryan, Siddhartha Jain, Mingyue Shang, Shiqi Wang, Xiaofei Ma, Murali Krishna Ramanathan, and Baishakhi Ray. 2024. Code-aware prompting: A study of coverage-guided test generation in regression setting using llm. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 951–971.
- [50] Edward K Smith, Earl T Barr, Claire Le Goues, and Yuriy Brun. 2015. Is the cure worse than the disease? overfitting in automated program repair. In *Proceedings of the 10th Joint Meeting on Foundations of Software Engineering*. 532–543.
- [51] Hongjin Su, Ruoxi Sun, Jinsung Yoon, Pengcheng Yin, Tao Yu, and Sercan Ö Arık. 2025. Learn-by-interact: A Data-Centric Framework For Self-Adaptive Agents in Realistic Environments. In *Proceedings of the Thirtieth International Conference on Learning Representations*.
- [52] Nikolai Tillmann and Jonathan De Halleux. 2008. Pex—white box test generation for. net. In *International conference on tests and proofs*. Springer, 134–153.

- [53] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, Shao Kun Deng, and Neel Sundaresan. 2020. Unit test case generation with transformers and focal context. *arXiv preprint arXiv:2009.05617* (2020).
- [54] Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. 2024. Openhands: An open platform for ai software developers as generalist agents. *arXiv preprint arXiv:2407.16741* (2024).
- [55] Zejun Wang, Kaibo Liu, Ge Li, and Zhi Jin. 2024. HITS: High-coverage LLM-based Unit Test Generation via Method Slicing. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1258–1268.
- [56] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025. Agentless: Demystifying llm-based software engineering agents. In *Proceedings of 33rd ACM SIGSOFT International Symposium on the Foundations of Software Engineering*.
- [57] Qi Xin and Steven P Reiss. 2017. Identifying test-suite-overfitted patches through test case generation. In *Proceedings of the 26th ACM SIGSOFT international symposium on software testing and analysis*. 226–236.
- [58] Yingfei Xiong, Xinyuan Liu, Muhan Zeng, Lu Zhang, and Gang Huang. 2018. Identifying patch correctness in test-based program repair. In *Proceedings of the 40th international conference on software engineering*. 789–799.
- [59] Chuyang Xu, Zhongxin Liu, Xiaoxue Ren, Gehao Zhang, Ming Liang, and David Lo. 2025. Flexfl: Flexible and effective fault localization with open-source large language models. *IEEE Transactions on Software Engineering* (2025).
- [60] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang (Eds.). http://papers.nips.cc/paper_files/paper/2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html
- [61] Jun Yang, Yuehan Wang, Yiling Lou, Ming Wen, and Lingming Zhang. 2023. A large-scale empirical review of patch correctness checking approaches. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1203–1215.
- [62] Zhongxing Yu, Matias Martinez, Benjamin Danglot, Thomas Durieux, and Martin Monperrus. 2019. Alleviating patch overfitting with automatic test generation: a study of feasibility and effectiveness for the nopol repair system. *Empirical Software Engineering* 24 (2019), 33–67.
- [63] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. Autocoderover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1592–1604.